

JDBC (Continue)

Prepared SQL

1. Overview

Each SQL statement you send to the database needs to be parsed by the database engine before it can actually be executed. When the database parses a SQL statement, it reads the SQL to determine what you want the database to do, and then it formulates a plan for carrying out your instructions. This processing is called building a query plan.

Each SQL statement you sent to the database required the database to treat the statement as a brand-new query and thus build a new query plan for it. This processing is necessary only if each statement requires a distinct query plan. If you are executing statements over and over again that have the same query plan, you are wasting processing power. If, for example, your banking application uses the SQL `UPDATE ACCOUNT SET BALANCE = XXX WHERE ACCOUNT_ID = YYY`, you would force the database to rebuild the same query plan each time you changed the balance for the account. Databases enable you to optimize repeated calls through prepared SQL.

Databases provide two kinds of prepared SQL: prepared statements and stored procedures. Prepared SQL provides an advantage over the simple SQL statements you have covered so far; a database can get the SQL ahead of time and create a query plan while you are doing other application logic. This means that your SQL should execute faster and that you can have a generic reference to a statement for later reuse rather than repeatedly create new SQL statements for each new access to the database.

2. Prepared Statements

The `PreparedStatement` interface extends the `Statement` interface you used in [Chapter 3](#). It enables a SQL statement to contain parameters like a function definition, and you can execute a single statement repeatedly with different values for those parameters. The act of assigning values to parameters is called *binding parameters*. You might want to use a prepared statement when updating a group of objects stored on the same table. For example, if you were updating many bank accounts at once, you might have a loop calling:

```

Statement statement = c.createStatement();
int i;

for(i=0; i<accounts.length; i++)
    statement.executeUpdate("UPDATE account " +
                           "SET balance = " +
accounts[i].getBalance( ) +
                           "WHERE id = " + accounts[i].getId( ));
c.commit( );
statement.close( );

```

This statement creates the same query plan each time through the loop. Instead of calling this same statement repeatedly with different inputs, you can instead use a `PreparedStatement`:

```

PreparedStatement statement = c.prepareStatement(
    "UPDATE account " +
    "SET balance = ? " +
    "WHERE id = ?");

int i;

for(i=0; i<accounts.length; i++) {
    statement.setFloat(1, accounts[i].getBalance( ));
    statement.setInt(2, accounts[i].getId( ));
    statement.execute( );
    statement.clearParameters( )
}
c.commit( );
statement.close( );

```

With a prepared statement, you send the actual SQL to the database when you get the `PreparedStatement` object through the `prepareStatement()` method in `java.sql.Connection`. Keep in mind that you have not yet actually executed any SQL. You execute that prepared SQL statement multiple times inside the `for()` loop, but you build the query plan only a single time.

Before each execution of the prepared statement, you tell JDBC which values to use as input for that execution of the statement. In order to bind the input parameters, `PreparedStatement` provides `setXXX()` methods (such as `setFloat()` and `setInt()`) that mirror the `getXXX()` methods you saw in `java.sql.ResultSet`. Just as the `getXXX()` methods read results according to the order in which you constructed your SQL, the `setXXX()` methods bind parameters from left to right in the order you placed them in the prepared statement. In the

previous example, I bound parameter 1 as a float to the account balance that I retrieved from the account object. The first ? was thus associated with parameter 1.

3. Stored Procedures

While prepared statements let you access similar database queries through a single `PreparedStatement` object, stored procedures attempt to take the “black box” concept for database access one step further. A stored procedure is built inside the database before you run your application. You access that stored procedure by name at runtime. In other words, a stored procedure is almost like a method you call in the database. Stored procedures have the following advantages:

- Because the procedure is precompiled in the database for most database engines, it executes much faster than dynamic SQL, which needs to be re-interpreted each time it is issued. Even if your database does not compile the procedure before it runs, it will be precompiled for subsequent runs just like prepared statements.
- Syntax errors in the stored procedure can be caught at compile time rather than at runtime.
- Java developers need to know only the name of the procedure and its inputs and outputs. The way in which the procedure is implemented—the tables it accesses, the structure of those tables, etc.—is completely unimportant.

A stored procedure is written with variables as argument place holders, which are passed when the procedure is called through column binding. Column binding is a fancy way of specifying the parameters to a stored procedure. You will see exactly how this is done in the following examples. A Sybase stored procedure might look like this:

```
DROP PROCEDURE sp_select_min_bal
GO

CREATE PROCEDURE sp_select_min_bal
    @balance,
AS
SELECT account_id
FROM account
WHERE balance > @balance

GO
```

The name of this stored procedure is `sp_select_min_bal`. It accepts a single argument identified by the `@` sign. That single argument is the minimum balance. The stored procedure produces a result set containing all accounts with a balance greater than that minimum balance. While this stored procedure produces a result set, you can also have procedures that return output parameters. Here's an even more complex stored procedure, written in Oracle's stored procedure language, that calculates interest and returns the new balance:

```
CREATE OR REPLACE PROCEDURE sp_interest
(id IN INTEGER,
bal IN OUT FLOAT) IS
BEGIN
SELECT balance
INTO bal
FROM account
WHERE account_id = id;

bal := bal + bal * 0.03;

UPDATE account
SET balance = bal
WHERE account_id = id;

END;
```

This stored procedure accepts two arguments—the variables in the parentheses—and does complex processing that does not (and cannot) occur in the embedded SQL you have been using so far. It actually performs two SQL statements and a calculation all in one procedure. The first part grabs the current balance; the second part takes the balance and increases it by 3 percent; and the third part updates the balance. In your Java application, you could use it like this:

```
try {
    CallableStatement statement;
    int i;

    statement = c.prepareCall("{call sp_interest[(?,?)]}");

    statement.registerOutParameter(2, java.sql.Types.FLOAT);
    for(i=1; i<accounts.length; i++) {
        statement.setInt(1, accounts[i].getId( ));
        statement.execute( );
        System.out.println("New balance: " +
statement.getFloat(2));
    }
    c.commit( );
}
```

```
statement.close( );
c.close( );
}
```

The `CallableStatement` class is very similar to the `PreparedStatement` class. Using `prepareCall( )` instead of `prepareStatement( )`, you indicate which procedure you want to call when you initialize your `CallableStatement` object. Unfortunately, this is one time when ANSI SQL2 simply is not enough for portability. Different database engines use different syntaxes for these calls. JDBC, however, does provide a database-independent, stored-procedure escape syntax in the form `{call procedure_name [(?, ?)]}`. For stored procedures with return values, the escape syntax is: `{? = call procedure_name [(?,?)]}`. In this escape syntax, each `?` represents a place holder for either procedure inputs or return values. The JDBC driver then translates this escape syntax into the driver's own stored procedure syntax.

If your stored procedure has output parameters, you need to register their types using `registerOutParameter( )` before executing the stored procedure. This step tells JDBC what datatype the parameter in question will be. The previous example did it like this:

```
CallableStatement statement;
int i;

statement = c.prepareCall("{call sp_interest[(?,?)]}");
statement.registerOutParameter(2, java.sql.Types.FLOAT);
```

The `prepareCall( )` method creates a stored procedure object that will make a call to the specified stored procedure. This syntax sets up the order you will use in binding parameters. By calling `registerOutParameter( )`, you tell the `CallableStatement` instance to expect the second parameter as output of type float. Once this is set up, you can bind the ID using `setInt( )`, and then get the result using `getFloat( )`.